COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010: Data Structures and Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	8 / 4 / 26	**DATE SUBMITTED:**	8 / 4 / 26
NAME:	Santos, Paterno Miguel R.	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 5.1: Queues

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A:

CODE

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> copyQ);

int main(){
    // Create an object
    std::queue<char> myQ;

    myQ.push('S');
    myQ.push('A');
    myQ.push('N');
    myQ.push('T');
    myQ.push('O');
    myQ.push('S');
    display(myQ);

    /* Try all member functions:
       Empty */
    std::cout << "The queue is ";
    if(myQ.empty()){
        std:: cout << "empty\n";
    }else{
        std:: cout << "not empty\n";
    }

    // Size
    std::cout << "The size of the queue is " << myQ.size() << std::endl;

    // Front
    std::cout << "The front of the queue is " << myQ.front() << std::endl;
    // Back
    std::cout << "The back of the queue is " << myQ.back() << std::endl;

    // Pop
    while(!myQ.empty()){
        myQ.pop();
    }

    // Display
    std::cout << "Entire queue has been popped!\n";
    std::cout << "Current queue: ";
    if(myQ.empty()){
        std::cout << "None \n";
    }
```

```
    // Push
    myQ.push('P');
    myQ.push('A');
    myQ.push('T');
    myQ.push('E');
    myQ.push('R');
    myQ.push('N');
    myQ.push('O');

    // Display
    std::cout << "The queue has been filled! \n";
    std::cout << "Current queue: ";
    display(myQ);

    return 0;
}

void display(std::queue<char> copyQ){
    // Creates a copy of the passed queue

    std::queue<char> c = copyQ;
    while(!c.empty()){
        std::cout << " " << c.front();
        c.pop();
    }
    std::cout<< "\n";
}
```

OUTPUT

```
 S A N T O S
The queue is not empty
The size of the queue is 6
The front of the queue is S
The back of the queue is S
Entire queue has been popped!
Current queue: None
The queue has been filled!
Current queue:  P A T E R N O
PS C:\Users\TIPQC\Documents\SANTOS\HOA5p1\ILO_A>
```

OBSERVATION

Using queues is very much the same as using stacks but the main difference is that it is first come, first out in queues while stacks is last come, first out. The commands almost works the same as using the commands with stacks.

ILO B:

CODE

main.cpp

```cpp
#include <iostream>
#include "queueLL.h"

int main(){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    enqueue('S', &front, &back);
    std::cout << front->data << " " << back->data << std::endl;
    enqueue('A', &front, &back);
    std::cout << front->data << " " << back->data << std::endl;
    enqueue('N', &front, &back);
    std::cout << front->data << " " << back->data << std::endl;
    enqueue('T', &front, &back);
    std::cout << front->data << " " << back->data << std::endl;
    enqueue('O', &front, &back);
    std::cout << front->data << " " << back->data << std::endl;
    enqueue('S', &front, &back);
    std::cout << front->data << " " << back->data << std::endl;

    dequeue(&front, &back);
    std::cout << front->data << " " << back->data << std::endl;
    dequeue(&front, &back);
    std::cout << front->data << " " << back->data << std::endl;

    Front(&front);
    Back(&back);
    displayAll(&front, &back);

    return 0;
}
```

queueLL.h

```cpp
#ifndef QUEUELL_H
#define QUEUELL_H

// Create a class the node
template <typename T>
class qNode{
    public:
        T data;
        qNode<T>* next;
};

// Creates the new node
template <typename T>
qNode<T>* new_node(T newData){
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

// Enqueue operation
template <typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
    // Create new node
    qNode<T>* newNode = new_node(newData);

    // Insertion to empty queue
    if((*frontPtr)==nullptr && (*backPtr)==nullptr){
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
    }

    // Insertion of item in non empty queue
    (*backPtr)->next = newNode;
    (*backPtr) = newNode;
}
```

```cpp
template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
    // Create temp node
    qNode<T>* deleteNode = nullptr; // temp var
    deleteNode = (*frontPtr);


    // Check if queue has only one node
    if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete deleteNode;
        return;
    }

    // Delete node
    (*frontPtr) = (*frontPtr)->next;
    deleteNode->next = nullptr;
    delete deleteNode;

}

// Check the front
template <typename T>
void Front(qNode<T>** frontPtr){
    if((*frontPtr) != nullptr){
        std::cout << "The front is " << (*frontPtr)->data << "\n";
    }
}
```

```
// Check the back
template <typename T>
void Back(qNode<T>** backPtr){
    if((*backPtr) != nullptr){
        std::cout << "The back is " << (*backPtr)->data << "\n";
    }
}


// Display All
template <typename T>
void displayAll(qNode<T>** frontPtr, qNode<T>** backPtr){
    if ((*frontPtr)==nullptr && (*backPtr)==nullptr) return;

    std::cout << "The current elements inside the queue: ";
    while ((*frontPtr) != nullptr) {
        std::cout << (*frontPtr)->data << " ";
        (*frontPtr) = (*frontPtr)->next;
    }
    std::cout << std::endl;

}


#endif
```

OUTPUT

```
S S
S A
S N
S T
S O
S S
A S
N S
The front is N
The back is S
The current elements inside the queue: N T O S
```

OBSERVATION

Recreating the commands by using a linked list allows me to only use needed commands and is more efficient when it comes to accessing the only needed commands I need by using templates that work like a linked list.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.